

Problem Statement

You are given an M

times N matrix filled with characters 'X' and 'O'. Your task is to capture all regions of 'O's that are "surrounded".

A region is captured by flipping all 'O's in that region to 'X's. A region of 'O's is considered surrounded if it is not connected to any 'O' on the border of the grid. A connection can be to an adjacent cell (up, down, left, or right).

The operation should be performed in-place on the given matrix.

Input Format

An M times N matrix of characters ('X' and 'O').

Output Format

The same matrix, with the surrounded 'O' regions flipped to 'X'.

Sample Case

Input Matrix:

```
[[ 'X', 'X', 'X', 'X', 'X'],
 [ 'X', 'O', 'O', 'X', 'O'],
 [ 'X', 'X', 'O', 'X', 'O'],
 [ 'X', 'O', 'X', 'O', 'X'],
 [ 'O', 'O', 'O', 'X', 'X']]
```

Expected Output (in-place modification):

```
[[ 'X', 'X', 'X', 'X', 'X'],
 [ 'X', 'O', 'O', 'X', 'O'],
 [ 'X', 'X', 'O', 'X', 'O'],
 [ 'X', 'O', 'X', 'X', 'X'],
 [ 'O', 'O', 'O', 'X', 'X']]
```

Explanation:

The 'O's on the border (e.g., at (4,0) and (1,4)) are "safe". Any region of 'O's connected to these safe 'O's is also safe.

The large cluster of 'O's in the bottom-left is connected to the 'O' at (4,0) and is therefore safe. The two 'O's on the right edge are on the border, so they are safe. The only 'O' that is not connected to any border 'O's is the one at (3,3). It forms a surrounded region of size 1 and is flipped to 'X'.

Code

```
#include<stdio.h>
#define max 20 // maximum grid size (20x20)

// Global variables
int m, n; // dimensions of the board
char board[max][max]; // board containing 'X' and 'O'
int visited[max][max]; // visited matrix to mark explored cells

// Recursive function to read the board input
void readMatrix(int i, int j){
    if(i == m){ // base case: finished all rows
        return;
    }
    if(j == n){ // move to next row when end of column is reached
        readMatrix(i+1,0);
        return;
    }

    scanf(" %c",&board[i][j]); // read each character ('X' or 'O')
    visited[i][j] = 0; // mark unvisited initially
    readMatrix(i,j+1); // move to next column
}

/*
DFS function to explore a region of connected 'O's.
- Stores cells of the region in cellX[] and cellY[].
- Keeps track of whether the region touches the border.
Returns 1 if connected to border, otherwise 0.
*/
int checkRegion(int x, int y, int *cellX, int *cellY, int *count){
    // boundary and stopping conditions
    if(x < 0 || x >= m || y < 0 || y >= n){
        return 0;
    }
    if(board[x][y] != 'O' || visited[x][y]){
        return 0;
    }

    // mark current cell visited
    visited[x][y] = 1;
    cellX[*count] = x;
    cellY[*count] = y;
    (*count)++;

    // check if this cell lies on the border
    int border = (x == 0 || x == m-1 || y == 0 || y == n-1);

    // explore all 4 neighbors (up, down, left, right)
    border |= checkRegion(x-1,y,cellX,cellY,count);
    border |= checkRegion(x+1,y,cellX,cellY,count);
    border |= checkRegion(x,y-1,cellX,cellY,count);
    border |= checkRegion(x,y+1,cellX,cellY,count);
}
```

```

    return border;
}

/*
Main solver:
- Iterate through the board.
- For every unvisited 'O', run DFS to get the whole region.
- If region does not touch border, flip all 'O's in it to 'X'.
*/
void solve(){
    for(int i = 0; i < m; i++){
        for(int j = 0; j < n; j++){
            if(board[i][j] == 'O' && !visited[i][j]){
                int cellX[max*max], cellY[max*max], count = 0;
                int touchBorder = checkRegion(i, j, cellX, cellY, &count);

                // Flip region only if it is completely surrounded
                if(!touchBorder){
                    for(int k = 0; k < count; k++){
                        board[cellX[k]][cellY[k]] = 'X';
                    }
                }
            }
        }
    }
}

// Recursive function to print the board
void printMatrix(int i, int j){
    if(i == m){ // all rows printed
        return;
    }
    if(j == n){ // move to next row
        printf("\n");
        printMatrix(i+1,0);
        return;
    }
    printf("%c", board[i][j]); // print character without extra space
    printMatrix(i,j+1);
}

int main(){
    scanf("%d %d",&m,&n); // read dimensions

    readMatrix(0,0); // read board values
    solve(); // process and flip surrounded regions
    printMatrix(0,0); // print final board

    return 0;
}

```

Submission Page

Grade

Reviewed on Sunday, 7 September 2025, 12:11 AM by Automatic grade

Grade: 10.00 / 10.00

Assessment report  [-]

[+] **Summary of tests**

 Submitted on Sunday, 7 September 2025, 12:11 AM ( Download)

3.c

```
1  #include<stdio.h>
2  #define max 20
3
4  int m,n;
5  char board[max][max];
6  int visited[max][max];
7
8  void readMatrix(int i, int j){
9      if(i == m){
10         return;
11     }
12     if(j == n){
13         readMatrix(i+1,0);
14         return;
15     }
16
17     scanf(" %c",&board[i][j]);
18     visited[i][j] = 0;
19     readMatrix(i,j+1);
20 }
21
22 int checkRegion(int x, int y, int *cellX, int *cellY, int *count){
23     if(x < 0 || x >= m || y<0 || y >= n){
24         return 0;
25     }
26     if(board[x][y] != '0' || visited[x][y]){
27         return 0;
28     }
29
30     visited[x][y] = 1;
31     cellX[*count] = x;
32     cellY[*count] = y;
33     (*count)++;
34
35     int border = (x == 0 || x == m-1 || y==0 || y== n-1);
36
37     border |= checkRegion(x-1,y,cellX,cellY,count);
38     border |= checkRegion(x+1,y,cellX,cellY,count);
39     border |= checkRegion(x,y-1,cellX,cellY,count);
40     border |= checkRegion(x,y+1,cellX,cellY,count);
41     return border;
```